

数据结构

Morning Flower and Evening Oath

XCPC vector 模板分册

目录

使用约定	iii
第一章 GNU 扩展可持久化序列	1
1.1 GNU rope 与版本共享	1
第二章 GNU 扩展有序树	2
2.1 PBDS 有序树与顺序统计	2
第三章 GNU 扩展哈希表	3
3.1 GNU 整数键哈希表	3
第四章 数据结构	4
4.1 并查集	4
4.2 可撤销并查集	5
4.3 树状数组与第 k 小	6
4.4 区间加与区间和	7
4.5 64 位异或线性基	8
第五章 带修改的莫队	11
5.1 带修改的莫队	11
第六章 带位置线性基	13
6.1 带位置限制的异或线性基	13
第七章 线性空间求交	15
7.1 异或线性空间求交	15
7.2 Zassenhaus: 同时求和空间与交空间	16
第八章 线段树常用修改	17
8.1 线段树: 区间乘、加、赋值与模意义求和	17
第九章 优化与可持久化	19
9.1 离散李超树与最优直线编号	19
9.2 主席树: 静态区间第 k 小	20
第十章 可持久化数组	23
10.1 可持久化数组: 历史版本分叉与单点修改	23
第十一章 可持久化区间操作	26
11.1 可持久化区间加与版本拼接	26
第十二章 动态区间顺序统计	29
12.1 动态区间第 k 小 (树状数组套权值线段树)	29
第十三章 区间不同数统计	32
13.1 静态区间不同数计数 (前驱位置主席树)	32

第十四章 随机平衡树	34
14.1 FHQ Treap：可重集合与顺序统计	34
14.2 隐式 Treap：序列翻转、加值与区间和	36
第十五章 伸展树	40
15.1 Splay：可重集合与顺序统计	40
第十六章 状态序列维护	44
16.1 状态序列 Treap：分组区间 GCD	44
第十七章 配套知识与验证范围	48
17.1 可持久化区间拼接的懒标记	48
17.2 杭州 2023 K：跨版本拼接应用	49
17.3 带修莫队：时间轴与区间状态	50
17.4 并查集模板题的接口范围	51
第一部分 跨类依赖	52
附录 A 数论	53
A.1 静态模整数	53

使用约定

C++20，默认包含 `bits/stdc++.h` 并使用 `std` 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册按“大类（分册）—种类（章）—项目（节）”组织。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 `vector` 代码。

第一章 GNU 扩展可持久化序列

1.1 GNU rope 与版本共享

GNU libstdc++ 扩展, 包含 ext/rope; 本工具链中随后重新包含 cassert, 保留断言接口。rp<int>、rp<char> 表示序列, 默认空。size()、empty()、push_back(x)、insert(p,x)、erase(p,len)、replace(p,x)、substr(p,len) 直接沿用 rope; 位置为 0-based, len 是长度而非右端点。约定 $0 \leq p \leq \text{size}$, 删除/截取要求 $\text{len} \leq \text{size} - p$, 读写单点要求 $p < \text{size}$ 。r[i] 读取第 i 项; substr 返回新序列。rp<T> b=a 共享底层树, 随后通过容器接口修改 b 不影响 a, 可用 vector<rp<int> 保存分支版本。复制构造共享根为 $O(1)$; 随机访问沿树路径, 正常平衡时 $O(\log n)$ 。插入/删除/截取涉及路径、叶块复制和可能的再平衡, 不能把每次操作都写成严格最坏 $O(\log n)$; 释放最后引用可能递归回收整棵子树。长期保留所有版本的内存不能按单个序列 $O(n)$ 估计。不可缓存修改前的迭代器或使用可变代理跨版本写入。仅使用默认内存分配器及普通整数/字符值; 函数生成型 rope 不在本模板范围。可持久化队列已通过定长构造加头尾下标版本: Q 个零单元, 复制根与 l/r, 入队 replace(r++,x), 出队读取 [l++]; 直接 push_back/erase 版本在针对数据上耗尽内存。线上 942 ms、591.26 MiB, 具体失败与通过范围见 docs/ROPE-NOTES.md。

```
1 template <class T> using rp = __gnu_cxx::rope<T>;
```

第二章 GNU 扩展有序树

2.1 PBDS 有序树与顺序统计

GNU libstdc++ 扩展, 包含 `ext/pb_ds/assoc_container.hpp` 和 `ext/pb_ds/tree_policy.hpp`。 `ost<T>` 为去重有序集合; `insert(x)`、`erase(x)`、`find(x)`、`lower_bound(x)`、`upper_bound(x)` 沿用集合接口。`order_of_key(x)` 为严格小于 `x` 的元素数, `find_by_order(k)` 返回 0-based 第 `k` 个的迭代器, 越界返回 `end`。默认 `less<T>`; 改用其他严格弱序比较器时, 排名和边界均按该比较器解释。单次 $O(\log n)$, 空间 $O(n)$, 假设比较为 $O(1)$ 。重复值使用 `ost<pair<T,int>`, 插入 `{x,id}` 且 `id` 全局唯一, $1 \leq id < INT_MAX$ 。`order_of_key({x,0})` 统计小于 `x`, `order_of_key({x,INT_MAX})` 统计不大于 `x`; `find_by_order(k)->first` 取值, 须先检查 `end`。删除一个 `x`: `it=lower_bound({x,0})`, 检查 `it!=end` 且 `it->first==x` 后 `erase(it)`。不可用 `less_equal` 模拟多重集合。前驱先检查迭代器是否为 `begin`, 再递减; 空集合和无解不得解引用 `end`。

```
1 template <class T, class C = less<T>>
2 using ost = __gnu_pbds::tree<T,
3                               __gnu_pbds::null_type,
4                               C,
5                               __gnu_pbds::rb_tree_tag,
6                               __gnu_pbds::tree_order_statistics_node_update>;
```

第三章 GNU 扩展哈希表

3.1 GNU 整数键哈希表

GNU libstdc++ 扩展, 包含 `ext/pb_ds/assoc_container.hpp`。 `gp_map<K,V>` 中 `K` 采用不超过 64 位的整数类型, `V` 为值类型; 默认空表。`a[k]` 读取或插入默认值, `find(k)` 查找且不插入, `insert({k,v})` 不覆盖已有键, `erase(k)`、`clear()`、`size()`、`empty()` 沿用容器接口。哈希使用进程内固定随机种子的 64 位混合; 算术为无符号回绕, 支持有符号整数转为 `uint64_t`。不保证抵抗已知种子的恶意构造。通常查改摊还 $O(1)$, 最坏单次 $O(n)$, 空间 $O(n)$; 不提供顺序统计。`rehash` 会使迭代器失效, 修改表后重新 `find`。默认策略不直接调用 `resize`, 不使用 `unordered_map` 的 `reserve` 接口。用于不存在时返回零的映射, 建议 `find` 后判断 `end`, 避免纯查询扩大表。

```
1 struct hash64
2 {
3     static uint64_t mix(uint64_t x)
4     {
5         x += 0x9e3779b97f4a7c15ULL;
6         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
7         x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
8         return x ^ (x >> 31);
9     }
10
11     size_t operator()(uint64_t x) const
12     {
13         static const uint64_t seed =
14             chrono::steady_clock::now().time_since_epoch().count();
15         return mix(x + seed);
16     }
17 };
18
19 template <class K, class V> using gp_map = __gnu_pbds::gp_hash_table<K, V, hash64>;
```

第四章 数据结构

4.1 并查集

`dsu(n)`, 点号 $0..n-1$, $n \geq 0$ 。 `find(x)` 路径减半; `same(x,y)` 判断同集合; `merge(x,y)` 把 y 的根接到 x 的根, 已同集合返回 `false`。 `size(x)` 返回所在集合大小。 `groups()` 返回所有非空分组, 组内点号递增、组间按当前根编号排列; 可能压缩路径, 但不改变分组。 `fa`、`sz` 可读, `sz` 仅根处有效; 不要外部改写。 本版只做路径减半, 不按大小或秩合并; 单次 `find` 最坏 $O(n)$ 。 空间 $O(n)$, `groups` 另需 $O(n)$ 输出与工作空间, 并执行 n 次 `find`。 空结构可调用 `groups`, 不能查询顶点。

```
1 struct dsu
2 {
3     vector<int> fa, sz;
4
5     dsu(int n) : fa(n), sz(n, 1) { iota(fa.begin(), fa.end(), 0); }
6
7     int find(int x)
8     {
9         while (x != fa[x]) x = fa[x] = fa[fa[x]];
10        return x;
11    }
12
13    bool same(int x, int y) { return find(x) == find(y); }
14
15    bool merge(int x, int y)
16    {
17        x = find(x), y = find(y);
18        if (x == y) return false;
19        sz[x] += sz[y], fa[y] = x;
20        return true;
21    }
22
23    int size(int x) { return sz[find(x)]; }
24
25    vector<vector<int>> groups()
26    {
27        vector<vector<int>> res(fa.size());
28        for (int i = 0; i < int(fa.size()); i++) res[find(i)].push_back(i);
29        res.erase(
30            remove_if(res.begin(), res.end(), [&](const auto &v) { return v.empty(); }),
31            res.end());
32        return res;
33    }
34};
```


4.2 可撤销并查集

构造后 `snapshot()` 保存历史长度, `rollback(t)` 回退。禁止路径压缩; `find/merge` 为 $O(\log n)$, 回退每次成功合并 $O(1)$ 。

```

1 struct RollbackDSU
2 {
3     vector<int> fa, siz;
4     vector<pair<int, int>> his;
5
6     RollbackDSU(int n) : fa(n + 1), siz(n + 1, 1) { iota(fa.begin(), fa.end(), 0); }
7
8     int find(int x) const
9     {
10         while (x != fa[x]) x = fa[x];
11         return x;
12     }
13
14     int snapshot() const { return (int)his.size(); }
15
16     bool merge(int x, int y)
17     {
18         x = find(x);
19         y = find(y);
20         if (x == y) return false;
21         if (siz[x] < siz[y]) swap(x, y);
22         his.push_back({y, siz[x]});
23         fa[y] = x;
24         siz[x] += siz[y];
25         return true;
26     }
27
28     void rollback(int t)
29     {
30         assert(0 <= t && t <= snapshot());
31         while (snapshot() > t)
32         {
33             auto [y, s] = his.back();
34             his.pop_back();
35             siz[fa[y]] = s;
36             fa[y] = y;
37         }
38     }
39 };

```

4.3 树状数组与第 k 小

$\text{add}(x,v)$ 、 $\text{sum}(x)$ 、 $\text{query}(l,r)$ ，下标 $1..n$ ，闭区间； $\text{kth}(k)$ 要求所有频率非负且 $k>0$ ，超过总和返回 $n+1$ 。单次 $O(\log n)$ 。T 须能从 0 构造并支持 $+=$ 与减法；可直接使用 `ModInt` 维护模数下的区间和。 kth 只适用于有序累计频数，不适用于取模后的区间和。

```

1 template <class T = long long> struct Fenwick
2 {
3     int n;
4     vector<T> a;
5
6     Fenwick(int n = 0) : n(n), a(n + 1) {}
7
8     void add(int x, T v)
9     {
10         assert(x >= 1);
11         for (; x <= n; x += x & -x) a[x] += v;
12     }
13
14     T sum(int x) const
15     {
16         T ans = 0;
17         for (; x; x -= x & -x) ans += a[x];
18         return ans;
19     }
20
21     T query(int l, int r) const { return sum(r) - sum(l - 1); }
22
23     // Nonnegative frequencies; returns n+1 if k exceeds total.
24     int kth(T k) const
25     {
26         assert(k > 0);
27         int x = 0, b = 1;
28         while (b <= n / 2) b *= 2;
29         for (; b >= 1)
30             if (x + b <= n && a[x + b] < k)
31             {
32                 k -= a[x + b];
33                 x += b;
34             }
35         return x + 1;
36     }
37 };

```

4.4 区间加与区间和

初值全零； $\text{add}(l,r,v)$ 、 $\text{query}(l,r)$ ， $1 \leq l \leq r \leq n$ 。维护 sum 与加法 tag ；所有中间和与长度乘积须在 long long 范围内。单次 $O(\log n)$ 。

```

1 struct LazySeg
2 {
3     int n;
4     vector<long long> sum, tag;
5
6     LazySeg(int n) : n(n), sum(4 * n + 4), tag(4 * n + 4) { assert(n > 0); }
7
8     void apply(int p, int l, int r, long long v)
9     {
10         sum[p] += (r - l + 1) * v;
11         tag[p] += v;
12     }
13
14     void push(int p, int l, int r)
15     {
16         int m = (l + r) / 2;
17         apply(p * 2, l, m, tag[p]);
18         apply(p * 2 + 1, m + 1, r, tag[p]);
19         tag[p] = 0;
20     }
21
22     void add(int l, int r, long long v) { add(1, 1, n, l, r, v); }
23
24     void add(int p, int l, int r, int ql, int qr, long long v)
25     {
26         if (ql <= l && r <= qr)
27         {
28             apply(p, l, r, v);
29             return;
30         }
31         push(p, l, r);
32         int m = (l + r) / 2;
33         if (ql <= m) add(p * 2, l, m, ql, qr, v);
34         if (qr > m) add(p * 2 + 1, m + 1, r, ql, qr, v);
35         sum[p] = sum[p * 2] + sum[p * 2 + 1];
36     }
37
38     long long query(int l, int r) { return query(1, 1, n, l, r); }
39
40     long long query(int p, int l, int r, int ql, int qr)
41     {
42         if (ql <= l && r <= qr) return sum[p];
43         push(p, l, r);
44         int m = (l + r) / 2;
45         long long ans = 0;
46         if (ql <= m) ans += query(p * 2, l, m, ql, qr);
47         if (qr > m) ans += query(p * 2 + 1, m + 1, r, ql, qr);
48         return ans;
49     }
50 };

```

4.5 64 位异或线性基

值类型 U 为 `unsigned long long`，完整支持 64 位。默认空基；`init/Init` 清空。`insert/Insert` 返回是否增秩；`contains/Contains` 判断可表示性（允许空子集，所以 0 总可表示）。`query/Query(x=0)` 返回 x 与所张成空间异或后的最大无符号值。`kth/Kth(k,nonempty=true)` 返回第 k 小的不同异或值，1-based，参数 k 为 `__uint128_t`，非法序号返回 `nullopt`。默认只允许非空输入子集，0 仅在 `dependent` 为真时出现；传 `false` 则包括空子集所得的 0。`minimum/Minimum` 同此约定，空输入的非空最小值不存在。`merge/Merge` 将另一基所代表的输入多重集并入；重复元素和自合并可新增异或为 0 的非空方案。`rank` 是维数，`dependent` 表示存在非空零异或子集。插入、成员判断、最大值为 $O(64)$ ；合并 $O(64^2)$ 。`kth` 会在需要时 `rebuild/Rebuild` 规范化缓存，首次有效查询 $O(64^2)$ ，后续 $O(64)$ ；仅新增独立向量使缓存失效。所有成员仅供只读，不要直接改数组或标志。

```

1 struct XorBasis
2 {
3     using U = unsigned long long;
4     using Wide = __uint128_t;
5     U a[64]{};
6     vector<U> p;
7     int rank = 0;
8     bool dependent = false, dirty = false;
9
10    void init()
11    {
12        fill(a, a + 64, 0);
13        p.clear();
14        rank = 0;
15        dependent = false;
16        dirty = false;
17    }
18
19    bool insert(U x)
20    {
21        for (int i = 63; i >= 0; i--)
22            if (x >> i & 1)
23            {
24                if (!a[i])
25                {
26                    a[i] = x;
27                    rank++;
28                    dirty = true;
29                    return true;
30                }
31                x ^= a[i];
32            }
33        dependent = true;
34        return false;
35    }

```

```
37     bool contains(U x) const
38     {
39         for (int i = 63; i >= 0; i--)
40             if (x >> i & 1) x ^= a[i];
41         return x == 0;
42     }
43
44     U query(U x = 0) const
45     {
46         for (int i = 63; i >= 0; i--) x = max(x, x ^ a[i]);
47         return x;
48     }
49
50     void rebuild()
51     {
52         vector<U> b(a, a + 64);
53         for (int i = 0; i < 64; i++)
54             if (b[i])
55                 for (int j = i + 1; j < 64; j++)
56                     if (b[j] >> i & 1) b[j] ^= b[i];
57         p.clear();
58         for (U x : b)
59             if (x) p.push_back(x);
60         dirty = false;
61     }
```

```

63 // k is 1-based; nonempty excludes the empty input subset, not value zero.
64 optional<U> kth(Wide k, bool nonempty = true)
65 {
66     if (!k) return nullopt;
67     Wide index = k;
68     if (!nonempty || dependent) index--;
69     if (index >= (Wide(1) << rank)) return nullopt;
70     if (dirty) rebuild();
71     U answer = 0;
72     for (int i = 0; i < rank; i++)
73         if (index >> i & 1) answer ^= p[i];
74     return answer;
75 }
76
77 optional<U> minimum(bool nonempty = true) { return kth(1, nonempty); }
78
79 void merge(const XorBasis &other)
80 {
81     dependent |= other.dependent;
82     for (U x : other.a)
83         if (x) insert(x);
84 }
85 };

```

第五章 带修改的莫队

5.1 带修改的莫队

ModifiedMo(n, m) 保存离线询问, $n > 0$, m 为修改总数。add(l, r, t) 登记第 t 次修改完成后的 $[l, r]$, 返回从 0 开始的询问编号; $0 \leq l \leq r \leq n$, $0 \leq t \leq m$, 询问数小于 INT_MAX。run(add, del, chg, ans, block=0) 排序并移动三个指针。add(p)/del(p) 按下标更新当前区间统计; chg($i, forward, l, r$) 将编号 i 的修改前进或回退, 并同步修正当前 $[l, r]$ 内的贡献; ans(id) 记录结果。修改编号从 0 开始, 前进第 i 次修改时原时间为 i , 回退时原时间为 $i+1$ 。回调初始状态必须为原数组、空区间、时间 0; 重复 run 前由调用方重置状态。状态不会自动恢复, 原询问顺序不变。仅适用于可增删维护的区间统计, 需区分左右端操作时另行拆分回调。默认块长约 $n^{2/3}$, 也可显式指定正块长。 n 、询问数、修改数同阶且回调 $O(1)$ 时总时间 $O(n^{5/3})$, 调度器空间 $O(q)$ 。

```
1 struct ModifiedMo
2 {
3     struct Query
4     {
5         int l, r, t, id;
6     };
7
8     int n, m;
9     vector<Query> q;
10
11     ModifiedMo(int n, int m) : n(n), m(m) { assert(n > 0 && m >= 0); }
12
13     int add(int l, int r, int t)
14     {
15         assert(0 <= l && l <= r && r <= n);
16         assert(0 <= t && t <= m);
17         int id = q.size();
18         q.push_back({l, r, t, id});
19         return id;
20     }
```

```

22  template <class Add, class Del, class Chg, class Ans>
23  void run(Add add, Del del, Chg chg, Ans ans, int block = 0) const
24  {
25      if (q.empty()) return;
26      if (block == 0) block = max(1, int(pow(n, 2.0 / 3)));
27      assert(block > 0);
28      auto ord = q;
29      auto cmp = [&](const Query &a, const Query &b)
30      {
31          int x = a.l / block, y = b.l / block;
32          if (x != y) return x < y;
33          int p = a.r / block, q = b.r / block;
34          if (p != q) return (x & 1) ? p > q : p < q;
35          return ((x ^ p) & 1) ? a.t > b.t : a.t < b.t;
36      };
37      sort(ord.begin(), ord.end(), cmp);
38      int l = 0, r = 0, t = 0;
39      for (auto q : ord)
40      {
41          while (t < q.t)
42          {
43              chg(t, true, l, r);
44              t++;
45          }
46          while (t > q.t)
47          {
48              t--;
49              chg(t, false, l, r);
50          }
51          while (l > q.l) add(--l);
52          while (r < q.r) add(r++);
53          while (l < q.l) del(l++);
54          while (r > q.r) del(--r);
55          ans(q.id);
56      }
57  }
58  };

```


第六章 带位置线性基

6.1 带位置限制的异或线性基

默认构造为空，init/Init 可清空。insert/Insert(x,at) 插入 unsigned long long 值及非负 int 标签，支持完整 64 位，标签可重复、乱序。contains/Contains(x,lower=0) 判断 x 能否由标签至少为 lower 的输入异或得到；query/Query(lower=0,x=0) 返回 x 与这些输入子集异或后的最大值；rank/Rank(lower=0) 返回对应空间维数。lower 为非负 long long，可大于所有 int 标签，空集仍可产生 0。全部操作 $O(64)$ ，空间 $O(64)$ 。消元时同一主元优先保留标签较大的向量，较旧向量继续消元；按阈值筛选后仍是正确线性基。标签是组合可用范围的界，不是选点方案；数组仅供只读。区间 [l,r] 应按右端点排序询问，扫描并仅插入到 r，再查询 lower=l；接口本身没有上界，插入超过 r 后不能只用 lower 恢复旧区间。若需保留历史，可复制每个前缀的对象，但空间为 $O(64n)$ 。不提供非空零异或判定或第 k 小；普通线性基的 dependent 不能直接移植到所有阈值。

```
1 struct PositionBasis
2 {
3     using U = unsigned long long;
4     U a[64];
5     int pos[64];
6
7     PositionBasis() { init(); }
8
9     void init()
10    {
11        fill(a, a + 64, 0);
12        fill(pos, pos + 64, -1);
13    }
```

```

15 void insert(U x, int at)
16 {
17     assert(at >= 0);
18     for (int i = 63; i >= 0; i--)
19         if (x >> i & 1)
20             {
21                 if (!a[i])
22                     {
23                         a[i] = x;
24                         pos[i] = at;
25                         return;
26                     }
27                 if (at > pos[i])
28                     {
29                         swap(a[i], x);
30                         swap(pos[i], at);
31                     }
32                 x ^= a[i];
33             }
34 }
35
36 bool contains(U x, long long lower = 0) const
37 {
38     assert(lower >= 0);
39     for (int i = 63; i >= 0; i--)
40         if ((x >> i & 1) && pos[i] >= lower) x ^= a[i];
41     return x == 0;
42 }
43
44 U query(long long lower = 0, U x = 0) const
45 {
46     assert(lower >= 0);
47     for (int i = 63; i >= 0; i--)
48         if (pos[i] >= lower) x = max(x, x ^ a[i]);
49     return x;
50 }
51
52 int rank(long long lower = 0) const
53 {
54     assert(lower >= 0);
55     int answer = 0;
56     for (int i = 0; i < 64; i++) answer += pos[i] >= lower;
57     return answer;
58 }
59 };

```

第七章 线性空间求交

7.1 异或线性空间求交

`basis_intersection(a,b)` 返回 `span(a)` 与 `span(b)` 的交空间的一组独立生成元，结果类型 `XorBasis`。支持完整 64 位无符号值，不修改输入，也不要求先 `rebuild`。使用第一个空间的主元，消去第二个空间的向量，同时记录来自第二个空间的部分；消元为零时，该部分属于交空间。时间 $O(64^2)$ ，额外空间 $O(64)$ 。输出按独立生成元解释：`dependent` 为 `false`，输入的重复次数及非空零异或关系不传播；表示的是子空间，不是两组原始输入元素的集合交。枚举整个交空间应调用 `result.kth(k,false)`，包含零；零空间返回空基，`rank=0`、`contains(0)=true`。

```
1 inline XorBasis basis_intersection(const XorBasis &a, const XorBasis &b)
2 {
3     using U = unsigned long long;
4     U pivot[64], from_b[64]{};
5     copy(a.a, a.a + 64, pivot);
6     XorBasis result;
7     for (U value : b.a)
8     {
9         U x = value, part = value;
10        for (int i = 63; i >= 0 && x; i--)
11            if (x >> i & 1)
12            {
13                if (!pivot[i])
14                {
15                    pivot[i] = x;
16                    from_b[i] = part;
17                    break;
18                }
19                x ^= pivot[i];
20                part ^= from_b[i];
21            }
22        if (!x && part) result.insert(part);
23    }
24    return result;
25 }
```

7.2 Zassenhaus: 同时求和空间与交空间

`basis_sum_intersection(a,b)` 返回和空间基、交空间基这一对结果。和空间由两组基共同张成，一般不是两个空间的集合并；两结果都是独立生成元，其 `dependent` 均为 `false`。将 `a` 的向量 `x` 编成 `(x,x)`，`b` 的向量 `y` 编成 `(y,0)`，用 `__uint128_t` 储存高低各 64 位并按高位到低位消元。高半部非零的行投影构成和空间的基，高半部为零的行取低半部构成交空间的基。必须先提升到 128 位再左移 64；不能用 64 位变量移位 64。时间 $O(128^2)$ ，额外空间 $O(128)$ 。结果满足 $\dim(A+B)+\dim(A \cap B)=\dim(A)+\dim(B)$ ，但维数恒等式只是检查条件，不能单独证明返回向量正确。包含空空间、相同空间和输入别名。

```

1 // Return independent generators of (sum space, intersection space).
2 inline pair<XorBasis, XorBasis> basis_sum_intersection(const XorBasis &a,
3                                                         const XorBasis &b)
4 {
5     using W = __uint128_t;
6     W pivot[128]{};
7     auto insert = [&](W x)
8     {
9         for (int i = 127; i >= 0 && x; i--)
10             if (x >> i & 1)
11             {
12                 if (!pivot[i])
13                 {
14                     pivot[i] = x;
15                     return;
16                 }
17                 x ^= pivot[i];
18             }
19     };
20     for (auto x : a.a) insert((W(x) << 64) | x);
21     for (auto x : b.a) insert(W(x) << 64);
22     XorBasis sum, intersection;
23     for (int i = 0; i < 128; i++)
24         if (pivot[i])
25         {
26             if (i < 64)
27                 intersection.insert((unsigned long long)pivot[i]);
28             else
29                 sum.insert((unsigned long long)(pivot[i] >> 64));
30         }
31     return {sum, intersection};
32 }

```

第八章 线段树常用修改

8.1 线段树：区间乘、加、赋值与模意义求和

`AffineSegTree(a,mod)` 传入非空 0-based 初值数组；查询与修改采用 1-based 非空闭区间。`update(l,r,mul,add)` 将区间元素 x 改为 $mul \cdot x + add$ ；乘法用 $(mul,0)$ ，加法用 $(1,add)$ ，赋值用 $(0,value)$ 。`query(l,r)` 返回区间和。 $1 \leq mod \leq INT_MAX$ ，模数无需为素数，初值和修改参数允许 `long long` 全范围，入口统一归一化。旧标记 (m,b) 后应用新标记 (a,c) ，复合为 $(m \cdot a, b \cdot a + c)$ ，乘法取模后再相加。建树 $O(n)$ ，单次 $O(\log n)$ ，空间 $O(n)$ 。内部 `build`、`apply` 等函数不作为公共接口；动态版重建新对象。

```
1 struct AffineSegTree
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         ll sum = 0, mul = 1, add = 0;
8     };
9
10    int n, mod;
11    vector<Node> t;
12
13    AffineSegTree(const vector<ll> &a, int mod) : n(a.size()), mod(mod), t(4 * n + 4)
14    {
15        assert(n > 0 && mod > 0);
16        build(1, 1, n, a);
17    }
18
19    ll norm(ll x) const
20    {
21        x %= mod;
22        if (x < 0) x += mod;
23        return x;
24    }
25
26    void pull(int p) { t[p].sum = (t[p * 2].sum + t[p * 2 + 1].sum) % mod; }
27
28    void build(int p, int l, int r, const vector<ll> &a)
29    {
30        t[p] = Node{};
31        if (l == r)
32        {
33            t[p].sum = norm(a[l - 1]);
34            return;
35        }
36        int m = (l + r) / 2;
37        build(p * 2, l, m, a);
38        build(p * 2 + 1, m + 1, r, a);
39        pull(p);
40    }
41
42    // New transform follows the old pending transform: x -> x*a+b.
```

```

43 void apply(int p, int len, ll a, ll b)
44 {
45     t[p].sum = (t[p].sum * a % mod + len * b % mod) % mod;
46     t[p].mul = t[p].mul * a % mod;
47     t[p].add = (t[p].add * a % mod + b) % mod;
48 }
49
50 void push(int p, int l, int r)
51 {
52     if (t[p].mul == 1 && t[p].add == 0) return;
53     int m = (l + r) / 2;
54     apply(p * 2, m - l + 1, t[p].mul, t[p].add);
55     apply(p * 2 + 1, r - m, t[p].mul, t[p].add);
56     t[p].mul = 1;
57     t[p].add = 0;
58 }
59
60 void update(int l, int r, ll a, ll b)
61 {
62     assert(1 <= l && l <= r && r <= n);
63     update(1, 1, n, l, r, norm(a), norm(b));
64 }
65
66 void update(int p, int l, int r, int ql, int qr, ll a, ll b)
67 {
68     if (ql <= l && r <= qr)
69     {
70         apply(p, r - l + 1, a, b);
71         return;
72     }
73     push(p, l, r);
74     int m = (l + r) / 2;
75     if (ql <= m) update(p * 2, l, m, ql, qr, a, b);
76     if (qr > m) update(p * 2 + 1, m + 1, r, ql, qr, a, b);
77     pull(p);
78 }
79
80 ll query(int l, int r)
81 {
82     assert(1 <= l && l <= r && r <= n);
83     return query(1, 1, n, l, r);
84 }
85
86 ll query(int p, int l, int r, int ql, int qr)
87 {
88     if (ql <= l && r <= qr) return t[p].sum;
89     push(p, l, r);
90     int m = (l + r) / 2;
91     ll answer = 0;
92     if (ql <= m) answer += query(p * 2, l, m, ql, qr);
93     if (qr > m) answer += query(p * 2 + 1, m + 1, r, ql, qr);
94     return answer % mod;
95 }
96 };

```

第九章 优化与可持久化

9.1 离散李超树与最优直线编号

构造传入所有可能查询的横坐标， $\text{add}(\{k,b,\text{id}\})$ 插入直线， $\text{query}(x)$ 返回最小值与最小编号，要求 x 在预登记坐标内且至少有一条线。整数求值用 `int128`， k 、 b 、 x 为有符号 64 位且中间值可表示。单次 $O(\log n)$ 。

```
1 struct LiChao
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     struct Line
7     {
8         ll k = 0, b = 0;
9         int id = -1;
10
11         I value(ll x) const { return I(k) * x + b; }
12     };
13
14     struct Node
15     {
16         Line line;
17         int left = -1, right = -1;
18     };
19
20     vector<ll> xs;
21     vector<Node> a{Node{}};
22
23     LiChao(vector<ll> x) : xs(x)
24     {
25         sort(xs.begin(), xs.end());
26         xs.erase(unique(xs.begin(), xs.end()), xs.end());
27         assert(!xs.empty());
28     }
29
30     bool better(Line u, Line v, ll x) const
31     {
32         return v.id == -1 ||
33             (u.id != -1 && pair{u.value(x), u.id} < pair{v.value(x), v.id});
34     }
35
36     void add(Line line)
37     {
38         assert(line.id >= 0);
39         add(0, 0, (int)xs.size() - 1, line);
40     }
41
42     void add(int p, int l, int r, Line line)
43     {
44         int m = (l + r) / 2;
45         if (better(line, a[p].line, xs[m])) swap(line, a[p].line);
```

```

46     if (l == r || line.id == -1) return;
47     bool left = better(line, a[p].line, xs[l]);
48     bool right = better(line, a[p].line, xs[r]);
49     if (!left && !right) return;
50     int next = left ? a[p].left : a[p].right;
51     if (next == -1)
52     {
53         next = (int)a.size();
54         a.push_back(Node{});
55         if (left)
56             a[p].left = next;
57         else
58             a[p].right = next;
59     }
60     if (left)
61         add(next, l, m, line);
62     else
63         add(next, m + 1, r, line);
64 }
65
66 pair<I, int> query(ll x) const
67 {
68     int idx = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
69     assert(idx < (int)xs.size() && xs[idx] == x);
70     int p = 0, l = 0, r = (int)xs.size() - 1;
71     Line best;
72     while (p != -1)
73     {
74         if (better(a[p].line, best, x)) best = a[p].line;
75         if (l == r) break;
76         int m = (l + r) / 2;
77         if (idx <= m)
78         {
79             p = a[p].left;
80             r = m;
81         }
82         else
83         {
84             p = a[p].right;
85             l = m + 1;
86         }
87     }
88     assert(best.id != -1);
89     return {best.value(x), best.id};
90 }
91 };

```

9.2 主席树：静态区间第 k 小

构造传入 0-based 数组， $\text{kth}(l, r, k)$ 的区间使用 1-based 闭区间；要求 $1 \leq k \leq r - l + 1$ 。离散化后每个前缀一个版本，单次 $O(\log n)$ ，总空间 $O(n \log n)$ 。

```

1 struct PersistentKth
2 {
3     struct Node

```



```

4      {
5          int l = 0, r = 0, sum = 0;
6      };
7
8      vector<Node> t{Node{}};
9      vector<int> root{0};
10     vector<long long> vals;
11
12     PersistentKth(const vector<long long> &a) : vals(a)
13     {
14         sort(vals.begin(), vals.end());
15         vals.erase(unique(vals.begin(), vals.end()), vals.end());
16         for (long long x : a)
17         {
18             int p = lower_bound(vals.begin(), vals.end(), x) - vals.begin();
19             root.push_back(add(root.back(), 0, (int)vals.size() - 1, p));
20         }
21     }
22
23     int add(int old, int l, int r, int x)
24     {
25         int p = t.size();
26         Node copy = t[old];
27         t.push_back(copy);
28         ++t[p].sum;
29         if (l < r)
30         {
31             int m = (l + r) / 2;
32             if (x <= m)
33             {
34                 int v = add(t[old].l, l, m, x);
35                 t[p].l = v;
36             }
37             else
38             {
39                 int v = add(t[old].r, m + 1, r, x);
40                 t[p].r = v;
41             }
42         }
43         return p;
44     }
45
46     long long kth(int l, int r, int k) const
47     {
48         assert(1 <= l && l <= r && r < (int)root.size() && 1 <= k && k <= r - l + 1);
49         int u = root[l - 1], v = root[r], L = 0, R = (int)vals.size() - 1;
50         while (L < R)
51         {
52             int m = (L + R) / 2, c = t[t[v].l].sum - t[t[u].l].sum;
53             if (k <= c)
54             {
55                 u = t[u].l;
56                 v = t[v].l;
57                 R = m;
58             }
59             else

```

```
60         {
61             k -= c;
62             u = t[u].r;
63             v = t[v].r;
64             L = m + 1;
65         }
66     }
67     return vals[L];
68 }
69 };
```

第十章 可持久化数组

10.1 可持久化数组：历史版本分叉与单点修改

PersistentArray(a,changes=0), a 是非空 0-based 初值数组; 第 0 版为初始状态。set(version,pos,value) 基于任意已有版本修改 1-based 位置, 创建并返回新版本编号; copy(version) 以 $O(1)$ 时间复制版本; query(version,pos) 只读且不自动创建版本。values 支持 long long 全范围。每次修改复制根到叶的节点, 旧版本不变; build 为内部函数, 不可覆盖历史节点。建树 $O(n)$, 修改及查询 $O(\log n)$, 初始 $2n-1$ 个节点, 每次修改至多 $\text{ceil}(\log_2(n))+1$ 个; 总空间 $O(n + \text{修改数} * \log n + \text{版本数})$ 。changes 仅用于预留节点池, 低估后可扩容, 所有外部引用应使用下标, 不保存 vector 元素引用。节点下标用 int, 节点总量须可表示。

```
1 struct PersistentArray
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0;
8         ll value = 0;
9     };
10
11     int n;
12     vector<Node> t{Node{}};
13     vector<int> root;
14
15     PersistentArray(const vector<ll> &a, int changes = 0) : n(a.size())
16     {
17         assert(n > 0 && changes >= 0);
18         int levels = 1;
19         for (int x = n - 1; x; x >>= 1) levels++;
20         t.reserve(2ULL * n + 1ULL * changes * levels + 1);
21         root.push_back(build(1, n, a));
22     }
23
24     int build(int l, int r, const vector<ll> &a)
25     {
26         int p = t.size();
27         t.push_back(Node{});
28         if (l == r)
29             t[p].value = a[l - 1];
30         else
31         {
32             int m = (l + r) / 2;
33             int left = build(l, m, a);
34             int right = build(m + 1, r, a);
35             t[p].l = left;
36             t[p].r = right;
37         }
38         return p;
39     }
40
41     int modify(int old, int l, int r, int pos, ll value)
```

```

42     {
43         int p = t.size();
44         Node copy = t[old];
45         t.push_back(copy);
46         if (l == r)
47             t[p].value = value;
48         else
49         {
50             int m = (l + r) / 2;
51             if (pos <= m)
52             {
53                 int child = modify(t[old].l, l, m, pos, value);
54                 t[p].l = child;
55             }
56             else
57             {
58                 int child = modify(t[old].r, m + 1, r, pos, value);
59                 t[p].r = child;
60             }
61         }
62         return p;
63     }
64
65     // Create a new version from any existing version. Positions are 1-based.
66     int set(int version, int pos, ll value)
67     {
68         assert(0 <= version && version < (int)root.size());
69         assert(1 <= pos && pos <= n);
70         int p = modify(root[version], 1, n, pos, value);
71         root.push_back(p);
72         return (int)root.size() - 1;
73     }
74
75     int copy(int version)
76     {
77         assert(0 <= version && version < (int)root.size());
78         root.push_back(root[version]);
79         return (int)root.size() - 1;
80     }
81
82     ll query(int version, int pos) const
83     {
84         assert(0 <= version && version < (int)root.size());
85         assert(1 <= pos && pos <= n);
86         int p = root[version], l = 1, r = n;
87         while (l < r)
88         {
89             int m = (l + r) / 2;
90             if (pos <= m)
91             {
92                 p = t[p].l;
93                 r = m;

```

```
94         }
95         else
96         {
97             p = t[p].r;
98             l = m + 1;
99         }
100     }
101     return t[p].value;
102 }
103 };
```

第十一章 可持久化区间操作

11.1 可持久化区间加与版本拼接

PersistentRange(n) 建立长度 n 的全零版本 0, $n \geq 1$; 使用隐式节点, 不预建整棵树。add(v,l,r,x) 在版本 v 的 $[l,r)$ 加 x ; splice(dst,src,l,r) 将 dst 的 $[l,r)$ 替换为 src 同一位置的值, 其他位置保留 dst; 二者均返回新版本编号。query(v,l,r) 返回区间和, const 且不分配节点。下标 0-based、区间半开, 允许空区间; 空修改及零增量仍创建版本但复用根, 整段拼接直接共享源根。全体版本属于同一对象、同一坐标域。懒标记保留在祖先, 用 offset 修正跨版本拼接; 每次修改/拼接/查询 $O(\log n)$, 版本别名 $O(1)$, 总节点 $O(q \log n)$ 。sum、lazy、delta、carry、offset 及其中间和差和长度乘积均须在 long long 内; 节点总数和版本数须小于 INT_MAX。t、root 可读以统计空间或查看根, 不要外部改写。

```
1 struct PersistentRange
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int left = 0, right = 0;
8         ll sum = 0, lazy = 0;
9     };
10
11     int n;
12     vector<Node> t{Node{}};
13     vector<int> root{0};
14
15     explicit PersistentRange(int size) : n(size) { assert(n > 0); }
16
17     int clone(int p)
18     {
19         Node node = t[p];
20         t.push_back(node);
21         return int(t.size()) - 1;
22     }
23
24     void pull(int p, int len)
25     {
26         t[p].sum = t[t[p].left].sum + t[t[p].right].sum + t[p].lazy * len;
27     }
```

区间加与版本拼接（接上页同一结构体）：

```

29     int change(int p, int l, int r, int ql, int qr, ll value)
30     {
31         if (qr <= l || r <= ql) return p;
32         int u = clone(p);
33         if (ql <= l && r <= qr)
34         {
35             t[u].lazy += value;
36             t[u].sum += value * (r - l);
37             return u;
38         }
39         int m = l + (r - l) / 2;
40         int a = change(t[p].left, l, m, ql, qr, value);
41         int b = change(t[p].right, m, r, ql, qr, value);
42         t[u].left = a;
43         t[u].right = b;
44         pull(u, r - l);
45         return u;
46     }
47
48     int replace(int p, int q, int l, int r, int ql, int qr, ll offset)
49     {
50         if (qr <= l || r <= ql) return p;
51         if (ql <= l && r <= qr)
52         {
53             if (offset == 0) return q;
54             int u = clone(q);
55             t[u].lazy += offset;
56             t[u].sum += offset * (r - l);
57             return u;
58         }
59         int u = clone(p), m = l + (r - l) / 2;
60         offset += t[q].lazy - t[p].lazy;
61         int a = replace(t[p].left, t[q].left, l, m, ql, qr, offset);
62         int b = replace(t[p].right, t[q].right, m, r, ql, qr, offset);
63         t[u].left = a;
64         t[u].right = b;
65         pull(u, r - l);
66         return u;
67     }

```

只读查询与对外接口（接上页同一结构体）：

```

69  ll get(int p, int l, int r, int ql, int qr, ll carry) const
70  {
71      if (qr <= l || r <= ql) return 0;
72      if (ql <= l && r <= qr) return t[p].sum + carry * (r - l);
73      carry += t[p].lazy;
74      int m = l + (r - l) / 2;
75      return get(t[p].left, l, m, ql, qr, carry) +
76             get(t[p].right, m, r, ql, qr, carry);
77  }
78
79  void check(int version, int l, int r) const
80  {
81      assert(0 <= version && version < int(root.size()));
82      assert(0 <= l && l <= r && r <= n);
83  }
84
85  int add(int version, int l, int r, ll value)
86  {
87      check(version, l, r);
88      int p = l == r || value == 0 ? root[version]
89              : change(root[version], 0, n, l, r, value);
90      root.push_back(p);
91      return int(root.size()) - 1;
92  }
93
94  int splice(int destination, int source, int l, int r)
95  {
96      check(destination, l, r);
97      check(source, l, r);
98      int p = l == r ? root[destination]
99              : replace(root[destination], root[source], 0, n, l, r, 0);
100     root.push_back(p);
101     return int(root.size()) - 1;
102 }
103
104 ll query(int version, int l, int r) const
105 {
106     check(version, l, r);
107     return l == r ? 0 : get(root[version], 0, n, l, r, 0);
108 }
109 };

```


第十二章 动态区间顺序统计

12.1 动态区间第 k 小（树状数组套权值线段树）

DynamicKth(a,future) 传入非空 0-based 初值, future 包含所有将来可能赋入的新值; 内部自动合并初值并离散化。set(pos,value) 单点赋值, kth(l,r,k) 查询闭区间第 k 小, 下标均为 1-based; $1 \leq k \leq r-l+1$ 。允许重复值与完整 long long 权值; 新值必须已在离散化集合内, 操作可按原顺序执行, 不保留历史版本。每个树状数组节点对应一棵可变权值线段树; 删除旧值后插入新值。空节点回收到 free_head 链, 重复修改不会按操作次数累计泄漏节点。设 D 为不同值数, $S=n+\text{future.size()}$, 预处理 $O(S \log S+n \log n \log(D+1))$, 单次修改及查询 $O(\log n \log(D+1))$ 。节点数上界 $B=(\text{ceil}(\log_2(D))+1) \cdot \sum \text{lowbit}(i)$, 求和 $i=1..n$; 空间 $O(S+n+B)$ 。update、modify、id 和节点池均为内部接口; 不能手动减去不存在的值。

```
1 struct DynamicKth
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, sum = 0;
8     };
9
10    int n, free_head = 0;
11    vector<int> bit, rank;
12    vector<ll> vals;
13    vector<Node> t{Node{}};
14
15    DynamicKth(const vector<ll> &a, vector<ll> future = {})
16        : n(a.size()), bit(n + 1), rank(n + 1)
17    {
18        assert(n > 0);
19        vals = future;
20        vals.insert(vals.end(), a.begin(), a.end());
21        sort(vals.begin(), vals.end());
22        vals.erase(unique(vals.begin(), vals.end()), vals.end());
23        for (int i = 1; i <= n; i++)
24        {
25            rank[i] = id(a[i - 1]);
26            modify(i, rank[i], 1);
27        }
28    }
29
30    int id(ll value) const
31    {
32        int p = lower_bound(vals.begin(), vals.end(), value) - vals.begin();
33        assert(p < (int)vals.size() && vals[p] == value);
34        return p;
35    }
36
37    int new_node()
38    {
39        if (!free_head)
```

```

40     {
41         t.push_back(Node{});
42         return (int)t.size() - 1;
43     }
44     int p = free_head;
45     free_head = t[p].l;
46     t[p] = Node{};
47     return p;
48 }
49
50 int update(int p, int l, int r, int x, int delta)
51 {
52     if (!p) p = new_node();
53     t[p].sum += delta;
54     if (l < r)
55     {
56         int m = (l + r) / 2;
57         if (x <= m)
58         {
59             int child = update(t[p].l, l, m, x, delta);
60             t[p].l = child;
61         }
62         else
63         {
64             int child = update(t[p].r, m + 1, r, x, delta);
65             t[p].r = child;
66         }
67     }
68     if (!t[p].sum)
69     {
70         t[p].l = free_head;
71         free_head = p;
72         return 0;
73     }
74     return p;
75 }
76
77 void modify(int pos, int x, int delta)
78 {
79     for (int i = pos; i <= n; i += i & -i)
80         bit[i] = update(bit[i], 0, (int)vals.size() - 1, x, delta);
81 }

```

```

83 void set(int pos, ll value)
84 {
85     assert(1 <= pos && pos <= n);
86     int x = id(value);
87     if (rank[pos] == x) return;
88     modify(pos, rank[pos], -1);
89     modify(pos, x, 1);
90     rank[pos] = x;
91 }
92
93 ll kth(int l, int r, int k) const
94 {
95     assert(1 <= l && l <= r && r <= n && 1 <= k && k <= r - l + 1);
96     vector<int> a, b;
97     for (int i = r; i; i -= i & -i) a.push_back(bit[i]);
98     for (int i = l - 1; i; i -= i & -i) b.push_back(bit[i]);
99     int L = 0, R = (int)vals.size() - 1;
100     while (L < R)
101     {
102         int count = 0;
103         for (int p : a) count += t[t[p].l].sum;
104         for (int p : b) count -= t[t[p].l].sum;
105         bool left = k <= count;
106         if (!left) k -= count;
107         for (int &p : a) p = left ? t[p].l : t[p].r;
108         for (int &p : b) p = left ? t[p].l : t[p].r;
109         int m = (L + R) / 2;
110         if (left)
111             R = m;
112         else
113             L = m + 1;
114     }
115     return vals[L];
116 }
117 };

```

第十三章 区间不同数统计

13.1 静态区间不同数计数（前驱位置主席树）

PersistentDistinct(a) 传入非空 0-based 数组，query(l,r) 返回闭区间内不同值的个数， $1 \leq l \leq r \leq n$ 。支持重复值与完整 long long 值域；建成后不支持改值，查询顺序任意。令 prev[i] 为同值上次出现的位置，未出现为 0；root[i] 在值域 0..n-1 中统计前 i 个 prev。区间内每种值恰有一次 prev[i]<l，因此答案为 root[r]-root[l-1] 中位置 0..l-1 的频数和。每个元素只复制一条节点路径，预处理 $O(n \log n)$ ，查询 $O(\log n)$ ，空间 $O(n \log n)$ 。动态版预留 $n \cdot (\text{ceil}(\log_2(n))+1)+1$ 个节点，避免扩容峰值；root 和节点池为内部数据。

```
1 struct PersistentDistinct
2 {
3     struct Node
4     {
5         int l = 0, r = 0, sum = 0;
6     };
7
8     int n;
9     vector<Node> t{Node{}};
10    vector<int> root;
11
12    PersistentDistinct(const vector<long long> &a) : n(a.size()), root(n + 1)
13    {
14        assert(n > 0);
15        int height = 1;
16        while ((1LL << (height - 1)) < n) height++;
17        t.reserve(size_t(n) * height + 1);
18        vector<long long> vals = a;
19        sort(vals.begin(), vals.end());
20        vals.erase(unique(vals.begin(), vals.end()), vals.end());
21        vector<int> last(vals.size());
22        for (int i = 1; i <= n; i++)
23        {
24            int x = lower_bound(vals.begin(), vals.end(), a[i - 1]) - vals.begin();
25            root[i] = insert(root[i - 1], 0, n - 1, last[x]);
26            last[x] = i;
27        }
28    }
29
30    int insert(int old, int l, int r, int x)
31    {
32        int p = t.size();
33        Node copy = t[old];
34        t.push_back(copy);
35        t[p].sum++;
36        if (l < r)
37        {
38            int m = (l + r) / 2;
39            if (x <= m)
40            {
41                int child = insert(t[old].l, l, m, x);
42                t[p].l = child;
```

```

43         }
44         else
45         {
46             int child = insert(t[old].r, m + 1, r, x);
47             t[p].r = child;
48         }
49     }
50     return p;
51 }
52
53 int query(int l, int r) const
54 {
55     assert(1 <= l && l <= r && r <= n);
56     int u = root[l - 1], v = root[r];
57     int L = 0, R = n - 1, answer = 0;
58     while (L < R)
59     {
60         int m = (L + R) / 2;
61         if (l - 1 <= m)
62         {
63             u = t[u].l;
64             v = t[v].l;
65             R = m;
66         }
67         else
68         {
69             answer += t[t[v].l].sum - t[t[u].l].sum;
70             u = t[u].r;
71             v = t[v].r;
72             L = m + 1;
73         }
74     }
75     return answer + t[v].sum - t[u].sum;
76 }
77 };

```

第十四章 随机平衡树

14.1 FHQ Treap: 可重集合与顺序统计

`insert(x)` 插入一个副本, `erase(x)` 删除一个副本并返回是否存在; `rank(x)` 为严格小于 `x` 的元素数加一, `x` 不必存在; `kth(k)` 要求 $1 \leq k \leq \text{size}()$ 。 `prev/next` 为严格前驱/后继, 不存在时返回 `nullopt`。支持重复值、`LLONG_MIN` 和 `LLONG_MAX`, 分裂不使用 `x-1`。随机优先级下单次期望 $O(\log n)$, 并非确定性最坏界。节点池不回收已删除节点, 空间按累计插入次数计算; 可在构造时指定随机种子。

```
1 struct OrderedTreap
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0;
8         ll val = 0;
9         unsigned long long pri = 0;
10    };
11
12    vector<Node> a{Node{}};
13    mt19937_64 rng;
14    int root = 0;
15
16    OrderedTreap(unsigned long long seed = 712367821) : rng(seed) {}
17
18    void pull(int p) { a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1; }
19
20    int node(ll x)
21    {
22        int p = a.size();
23        a.push_back({0, 0, 1, x, rng()});
24        return p;
25    }
26
27    // left: val < x, or val <= x when equal is true. No x-1 overflow.
28    void split(int p, ll x, int &l, int &r, bool equal = false)
29    {
30        if (!p)
31        {
32            l = 0;
33            r = 0;
34            return;
35        }
36        if (a[p].val < x || (equal && a[p].val == x))
37        {
38            l = p;
39            split(a[p].r, x, a[p].r, r, equal);
40        }
41        else
42        {
43            r = p;
```

```

44         split(a[p].l, x, l, a[p].l, equal);
45     }
46     pull(p);
47 }
48
49 int merge(int l, int r)
50 {
51     if (!l || !r) return l ? l : r;
52     if (a[l].pri > a[r].pri)
53     {
54         a[l].r = merge(a[l].r, r);
55         pull(l);
56         return l;
57     }
58     a[r].l = merge(l, a[r].l);
59     pull(r);
60     return r;
61 }
62
63 void insert(ll x)
64 {
65     int p = node(x), l, r;
66     split(root, x, l, r);
67     root = merge(merge(l, p), r);
68 }
69
70 bool erase(ll x)
71 {
72     int l, mid, r;
73     split(root, x, l, mid);
74     split(mid, x, mid, r, true);
75     bool found = mid != 0;
76     if (mid) mid = merge(a[mid].l, a[mid].r);
77     root = merge(l, merge(mid, r));
78     return found;
79 }
80
81 int size() const { return a[root].siz; }
82
83 int less(ll x, bool equal = false) const
84 {
85     int p = root, ans = 0;
86     while (p)
87     {
88         if (a[p].val < x || (equal && a[p].val == x))
89         {
90             ans += a[a[p].l].siz + 1;
91             p = a[p].r;
92         }
93         else
94             p = a[p].l;
95     }
96     return ans;
97 }
98
99 int rank(ll x) const { return less(x) + 1; }

```

```

100
101 ll kth(int k) const
102 {
103     assert(1 <= k && k <= size());
104     int p = root;
105     while (true)
106     {
107         int left = a[p].l].siz;
108         if (k == left + 1) return a[p].val;
109         if (k <= left)
110             p = a[p].l;
111         else
112         {
113             k -= left + 1;
114             p = a[p].r;
115         }
116     }
117 }
118
119 optional<ll> prev(ll x) const
120 {
121     int k = less(x);
122     if (!k) return nullopt;
123     return kth(k);
124 }
125
126 optional<ll> next(ll x) const
127 {
128     int k = less(x, true);
129     if (k == size()) return nullopt;
130     return kth(k + 1);
131 }
132 };

```

14.2 隐式 Treap：序列翻转、加值与区间和

insert(k,value) 在前 k 个元素后插入， $0 \leq k \leq \text{size}()$ ；erase(l,r)、reverse(l,r)、add(l,r,value)、query(l,r) 均使用 1-based 非空闭区间。values() 返回当前完整序列， $O(n)$ ，使用递归中序遍历。其余操作在随机优先级下期望 $O(\log n)$ ；翻转与加法懒标记同时维护。所有值、和、标记及 $\text{value} \times \text{长度}$ 中间结果必须落在 long long 范围内。删除不回收节点，内存按累计插入数计算。

```

1 struct SequenceTreap
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0;
8         ll val = 0, sum = 0, tag = 0;
9         unsigned long long pri = 0;
10        bool rev = false;
11    };
12
13    vector<Node> a{Node{}};

```



```

14  mt19937_64 rng;
15  int root = 0;
16
17  SequenceTreap(unsigned long long seed = 712367821) : rng(seed) {}
18
19  int size() const { return a[root].siz; }
20
21  void pull(int p)
22  {
23      a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1;
24      a[p].sum = a[a[p].l].sum + a[p].val + a[a[p].r].sum;
25  }
26
27  void apply_add(int p, ll x)
28  {
29      if (!p) return;
30      a[p].val += x;
31      a[p].sum += x * a[p].siz;
32      a[p].tag += x;
33  }
34
35  void apply_reverse(int p)
36  {
37      if (!p) return;
38      swap(a[p].l, a[p].r);
39      a[p].rev = !a[p].rev;
40  }
41
42  void push(int p)
43  {
44      if (a[p].rev)
45      {
46          apply_reverse(a[p].l);
47          apply_reverse(a[p].r);
48          a[p].rev = false;
49      }
50      if (a[p].tag)
51      {
52          apply_add(a[p].l, a[p].tag);
53          apply_add(a[p].r, a[p].tag);
54          a[p].tag = 0;
55      }
56  }
57
58  // Split into the first k elements and the rest.
59  void split(int p, int k, int &l, int &r)
60  {
61      if (!p)
62      {
63          l = 0;
64          r = 0;
65          return;
66      }
67      push(p);
68      int left = a[a[p].l].siz;
69      if (k > left)

```

```

70     {
71         l = p;
72         split(a[p].r, k - left - 1, a[p].r, r);
73     }
74     else
75     {
76         r = p;
77         split(a[p].l, k, l, a[p].l);
78     }
79     pull(p);
80 }
81
82 int merge(int l, int r)
83 {
84     if (!l || !r) return l ? l : r;
85     if (a[l].pri > a[r].pri)
86     {
87         push(l);
88         a[l].r = merge(a[l].r, r);
89         pull(l);
90         return l;
91     }
92     push(r);
93     a[r].l = merge(l, a[r].l);
94     pull(r);
95     return r;
96 }
97
98 // Insert after the first k elements, 0 <= k <= size().
99 void insert(int k, ll value)
100 {
101     assert(0 <= k && k <= size());
102     int p = a.size(), l, r;
103     a.push_back({0, 0, 1, value, value, 0, rng(), false});
104     split(root, k, l, r);
105     root = merge(merge(l, p), r);
106 }
107
108 // All range operations use 1-based inclusive intervals.
109 void erase(int l, int r)
110 {
111     assert(1 <= l && l <= r && r <= size());
112     int x, y, z;
113     split(root, r, y, z);
114     split(y, l - 1, x, y);
115     root = merge(x, z);
116 }
117
118 void reverse(int l, int r)
119 {
120     assert(1 <= l && l <= r && r <= size());
121     int x, y, z;
122     split(root, r, y, z);
123     split(y, l - 1, x, y);
124     apply_reverse(y);
125     root = merge(merge(x, y), z);

```

```

126     }
127
128     void add(int l, int r, ll value)
129     {
130         assert(1 <= l && l <= r && r <= size());
131         int x, y, z;
132         split(root, r, y, z);
133         split(y, l - 1, x, y);
134         apply_add(y, value);
135         root = merge(merge(x, y), z);
136     }
137
138     ll query(int l, int r)
139     {
140         assert(1 <= l && l <= r && r <= size());
141         int x, y, z;
142         split(root, r, y, z);
143         split(y, l - 1, x, y);
144         ll ans = a[y].sum;
145         root = merge(merge(x, y), z);
146         return ans;
147     }
148
149     void collect(int p, vector<ll> &out)
150     {
151         if (!p) return;
152         push(p);
153         collect(a[p].l, out);
154         out.push_back(a[p].val);
155         collect(a[p].r, out);
156     }
157
158     vector<ll> values()
159     {
160         vector<ll> answer;
161         collect(root, answer);
162         return answer;
163     }
164 };

```

第十五章 伸展树

15.1 Splay: 可重集合与顺序统计

insert/erase/rank/kth/prev/next 与有序 Treap 同义, 严格前驱后继不存在返回 nullopt。重复键计数存储; 失败查询也伸展最后访问节点, 单次均摊 $O(\log n)$ 。不回收节点, 空间按累计新建节点数计算。支持 long long 全范围; 不包含序列 Splay、虚子树信息或矩阵动态 DP。

```
1 struct OrderedSplay
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int ch[2]{}, fa = 0, cnt = 0, siz = 0;
8         ll val = 0;
9     };
10
11     vector<Node> a{Node{}};
12     int root = 0;
13
14     int size() const { return a[root].siz; }
15
16     void pull(int x) { a[x].siz = a[a[x].ch[0]].siz + a[x].cnt + a[a[x].ch[1]].siz; }
17
18     int node(ll value, int parent)
19     {
20         int x = a.size();
21         a.push_back({{0, 0}, parent, 1, 1, value});
22         return x;
23     }
24
25     void rotate(int x)
26     {
27         int y = a[x].fa, z = a[y].fa;
28         int k = a[y].ch[1] == x;
29         int w = a[x].ch[k ^ 1];
30         if (z)
31             a[z].ch[a[z].ch[1] == y] = x;
32         else
33             root = x;
34         a[x].fa = z;
35         a[y].ch[k] = w;
36         if (w) a[w].fa = y;
37         a[x].ch[k ^ 1] = y;
38         a[y].fa = x;
39         pull(y);
40         pull(x);
41     }
42
43     void splay(int x)
44     {
```

```

45     while (a[x].fa)
46     {
47         int y = a[x].fa, z = a[y].fa;
48         if (z)
49         {
50             bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
51             rotate(same ? y : x);
52         }
53         rotate(x);
54     }
55 }
56
57 // Splay the last visited node even when the value does not exist.
58 int find(ll value)
59 {
60     int x = root, last = 0;
61     while (x && a[x].val != value)
62     {
63         last = x;
64         x = a[x].ch[value > a[x].val];
65     }
66     if (x)
67         splay(x);
68     else if (last)
69         splay(last);
70     return x;
71 }
72
73 void insert(ll value)
74 {
75     int x = root, parent = 0;
76     while (x && a[x].val != value)
77     {
78         parent = x;
79         x = a[x].ch[value > a[x].val];
80     }
81     if (x)
82     {
83         ++a[x].cnt;
84         pull(x);
85     }
86     else
87     {
88         x = node(value, parent);
89         if (parent)
90             a[parent].ch[value > a[parent].val] = x;
91         else
92             root = x;
93     }
94     splay(x);
95 }
96
97 bool erase(ll value)
98 {
99     int x = find(value);
100    if (!x) return false;

```

```

101     if (a[x].cnt > 1)
102     {
103         --a[x].cnt;
104         pull(x);
105         return true;
106     }
107     int l = a[x].ch[0], r = a[x].ch[1];
108     if (l) a[l].fa = 0;
109     if (r) a[r].fa = 0;
110     if (!l)
111     {
112         root = r;
113         return true;
114     }
115     root = l;
116     x = l;
117     while (a[x].ch[1]) x = a[x].ch[1];
118     splay(x);
119     a[x].ch[1] = r;
120     if (r) a[r].fa = x;
121     pull(x);
122     return true;
123 }
124
125 int rank(ll value)
126 {
127     int x = root, last = 0, ans = 1;
128     while (x)
129     {
130         last = x;
131         if (value <= a[x].val)
132             x = a[x].ch[0];
133         else
134         {
135             ans += a[a[x].ch[0]].siz + a[x].cnt;
136             x = a[x].ch[1];
137         }
138     }
139     if (last) splay(last);
140     return ans;
141 }
142
143 ll kth(int k)
144 {
145     assert(1 <= k && k <= size());
146     int x = root;
147     while (true)
148     {
149         int left = a[a[x].ch[0]].siz;
150         if (k <= left)
151             x = a[x].ch[0];
152         else if (k <= left + a[x].cnt)
153         {
154             splay(x);
155             return a[x].val;
156         }

```

```

157         else
158         {
159             k -= left + a[x].cnt;
160             x = a[x].ch[1];
161         }
162     }
163 }
164
165 optional<ll> prev(ll value)
166 {
167     int x = root, last = 0, answer = 0;
168     while (x)
169     {
170         last = x;
171         if (a[x].val < value)
172         {
173             answer = x;
174             x = a[x].ch[1];
175         }
176         else
177             x = a[x].ch[0];
178     }
179     if (last) splay(last);
180     if (!answer) return nullopt;
181     splay(answer);
182     return a[answer].val;
183 }
184
185 optional<ll> next(ll value)
186 {
187     int x = root, last = 0, answer = 0;
188     while (x)
189     {
190         last = x;
191         if (a[x].val > value)
192         {
193             answer = x;
194             x = a[x].ch[0];
195         }
196         else
197             x = a[x].ch[1];
198     }
199     if (last) splay(last);
200     if (!answer) return nullopt;
201     splay(answer);
202     return a[answer].val;
203 }
204 };

```

第十六章 状态序列维护

16.1 状态序列 Treap：分组区间 GCD

build(v) 用数值与状态组成的序列替换当前内容。数值为无符号 64 位，状态为 0 或 1。insert(k,value,tag) 在前 k 项后插入。erase(l,r)、query(l,r,tag) 使用 1-based 非空闭区间；set(k,value) 修改值，toggle(k) 翻转状态。query 返回选中状态的 GCD，无对应元素为 nullopt，实际全零则返回 0。回收删除节点，空间按峰值长度计算。建树 $O(n)$ 次 GCD，随机优先级下单次期望 $O(\log n)$ 次 GCD；删除另需 $O(\text{删除长度})$ 回收。一次 GCD 为 $O(\log U)$ 。覆盖 kuangbin 3.7 的序列操作，尚无该原题 OJ 记录。建树后的聚合与删除节点回收均使用递归遍历；笛卡尔树的构造单调栈保留。

```
1 struct GcdSequenceTreap
2 {
3     using U = unsigned long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0, tag = 0;
8         int cnt[2]{};
9         U val = 0, g[2]{}, pri = 0;
10    };
11
12    vector<Node> a{Node{}};
13    vector<int> recycled;
14    mt19937_64 rng;
15    int root = 0;
16
17    GcdSequenceTreap(U seed = 712367821) : rng(seed) {}
18
19    int size() const { return a[root].siz; }
20
21    int node(U value, int tag)
22    {
23        assert(tag == 0 || tag == 1);
24        int p;
25        if (recycled.empty())
26        {
27            p = a.size();
28            a.push_back(Node{});
29        }
30        else
31        {
32            p = recycled.back();
33            recycled.pop_back();
34            a[p] = Node{};
35        }
36        a[p].siz = 1;
37        a[p].val = value;
38        a[p].tag = tag;
39        a[p].cnt[tag] = 1;
40        a[p].g[tag] = value;
41        a[p].pri = rng();
```



```

42     return p;
43 }
44
45 void pull(int p)
46 {
47     int l = a[p].l, r = a[p].r;
48     a[p].siz = a[l].siz + a[r].siz + 1;
49     for (int t = 0; t < 2; t++)
50     {
51         a[p].cnt[t] = a[l].cnt[t] + a[r].cnt[t] + (a[p].tag == t);
52         a[p].g[t] = gcd(a[l].g[t], a[r].g[t]);
53         if (a[p].tag == t) a[p].g[t] = gcd(a[p].g[t], a[p].val);
54     }
55 }
56
57 void split(int p, int k, int &l, int &r)
58 {
59     if (!p)
60     {
61         l = 0;
62         r = 0;
63         return;
64     }
65     int left = a[a[p].l].siz;
66     if (k > left)
67     {
68         l = p;
69         split(a[p].r, k - left - 1, a[p].r, r);
70     }
71     else
72     {
73         r = p;
74         split(a[p].l, k, l, a[p].l);
75     }
76     pull(p);
77 }
78
79 int merge(int l, int r)
80 {
81     if (!l || !r) return l ? l : r;
82     if (a[l].pri > a[r].pri)
83     {
84         a[l].r = merge(a[l].r, r);
85         pull(l);
86         return l;
87     }
88     a[r].l = merge(l, a[r].l);
89     pull(r);
90     return r;
91 }
92
93 void rebuild(int p)
94 {
95     if (!p) return;
96     rebuild(a[p].l);
97     rebuild(a[p].r);

```

```

98     pull(p);
99 }
100
101 void release(int p)
102 {
103     if (!p) return;
104     release(a[p].l);
105     release(a[p].r);
106     recycled.push_back(p);
107 }
108
109 // O(n) GCD calls; replaces the current sequence.
110 void build(const vector<pair<U, int>> &v)
111 {
112     a.assign(1, Node{});
113     recycled.clear();
114     root = 0;
115     vector<int> st;
116     for (auto [value, tag] : v)
117     {
118         int p = node(value, tag), last = 0;
119         while (!st.empty() && a[st.back()].pri < a[p].pri)
120         {
121             last = st.back();
122             st.pop_back();
123         }
124         a[p].l = last;
125         if (!st.empty()) a[st.back()].r = p;
126         st.push_back(p);
127     }
128     if (st.empty()) return;
129     root = st.front();
130     rebuild(root);
131 }
132
133 // Insert after the first k entries. Range operations below are 1-based.
134 void insert(int k, U value, int tag)
135 {
136     assert(0 <= k && k <= size());
137     int p = node(value, tag), l, r;
138     split(root, k, l, r);
139     root = merge(merge(l, p), r);
140 }
141
142 void erase(int l, int r)
143 {
144     assert(1 <= l && l <= r && r <= size());
145     int x, y, z;
146     split(root, r, y, z);
147     split(y, l - 1, x, y);
148     root = merge(x, z);
149     release(y);
150 }
151
152 void set(int k, U value)
153 {

```

```

154     assert(1 <= k && k <= size());
155     int x, y, z;
156     split(root, k, y, z);
157     split(y, k - 1, x, y);
158     a[y].val = value;
159     pull(y);
160     root = merge(merge(x, y), z);
161 }
162
163 void toggle(int k)
164 {
165     assert(1 <= k && k <= size());
166     int x, y, z;
167     split(root, k, y, z);
168     split(y, k - 1, x, y);
169     a[y].tag ^= 1;
170     pull(y);
171     root = merge(merge(x, y), z);
172 }
173
174 optional<U> query(int l, int r, int tag)
175 {
176     assert(1 <= l && l <= r && r <= size());
177     assert(tag == 0 || tag == 1);
178     int x, y, z;
179     split(root, r, y, z);
180     split(y, l - 1, x, y);
181     optional<U> answer;
182     if (a[y].cnt[tag]) answer = a[y].g[tag];
183     root = merge(merge(x, y), z);
184     return answer;
185 }
186 };

```

第十七章 配套知识与验证范围

17.1 可持久化区间拼接的懒标记

代码见第 26 页。对象从全零版本开始，同一对象内版本共享坐标域。区间加仅复制被访问节点，不向旧子节点下传标记。节点的和满足 $\text{sum} = \text{left.sum} + \text{right.sum} + \text{lazy} \cdot \text{len}$ 。读取时把祖先累积增量作为参数传下去，因此查询完全只读。

拼接含义为将目标版本某个半开区间替换成源版本同一坐标区间。由于两棵树祖先的 lazy 可能不同，不能直接将源子树挂到目标祖先下面。递归参数 offset 保持“源祖先增量减目标祖先增量”；下降时更新为 $\text{offset} + \text{source.lazy} - \text{destination.lazy}$ 。当区间完整覆盖，复制源节点并将这个 offset 加到其 lazy 和 sum，此后挂到目标祖先下面时，逻辑值才与源版本相同。offset 为零时直接共享源节点；没有覆盖时保留目标节点。空区间、零增量与全区间拼接都可仅增加版本别名。

每一层至多有两个边界节点，其他分支直接返回或复制一个整段节点，所以单次修改、拼接均为 $O(\log n)$ 新节点，查询为 $O(\log n)$ 时间、零新节点。不支持不同坐标偏移的平移粘贴，也不回收仍被旧版本引用的节点。所有 long long 的和、差、懒标记与长度乘积都应在范围内；节点数与版本数须小于 INT_MAX。

验证用独立向量保存各分支版本，逐区间检查更新、拼接和历史版本；包含正负增量、不同祖先标记、空区间、全段共享，以及查询前后的节点数量检查。规模验证包含长度 200000 的 10000 次区间版本组合，并在长度 INT_MAX 的隐式坐标域验证端点操作与共享根。普通与 ASan/UBSan 均通过；区间加 1、后缀拼接和单点读取的应用已获 QOJ8240 双 AC，其余通用接口范围仍由本地验证覆盖。来源需求是 2023 杭州站题解 第 24-26 页的 K 题，完整驱动及验证见下节。

17.2 杭州 2023 K：跨版本拼接应用

[Card Game](#) 的完整驱动在 `verify/qoj/8240/*.cpp`，使用前节的可持久化区间模板。将下标转为从 0 开始，设 $F_i(r)$ 为区间 $[i, r]$ 的剩余牌数， p 为 i 之后第一张与 a_i 相同的牌，若不存在则 $p = n$ 。当 $r < p$ 时，第一张牌始终保留，剩余牌的过程与从 $i + 1$ 开始相同；当 $r \geq p$ 时，到达 p 会将此前区间全部清空。因此

$$F_i(r) = \begin{cases} 1 + F_{i+1}(r), & r < p, \\ F_{p+1}(r), & r \geq p. \end{cases}$$

每个版本的坐标是右端点 r ，值是 $F_i(r)$ 。从 $i = n - 1$ 向前处理：复制版本 $i + 1$ 并在 $[i, p)$ 加 1，再把 $[p, n)$ 拼接自版本 $p + 1$ 。 $p = n$ 时省去后一步。询问原问题的 $[l, r]$ 时，从版本 $l - 1$ 读取坐标 $r - 1$ 的单点值。端点先与上次答案异或解码，不能将输入查询预先离线排序。

$n, q \leq 300000$ ，牌号位于 $[1, n]$ 。构建及查询时间分别为 $O(n \log n)$ 、 $O(q \log n)$ 。驱动另计算各次操作可能访问的相交节点数作为分配上界，一次 reserve 足够容纳整个构建过程，避免复制扩容的临时峰值。该计数仅依据区间结构，不影响牌局答案，且同样为 $O(n \log n)$ 。

本地参照按题意逐张放牌，遇重复就实际删除栈中相应后缀。穷举长度不超过 7 的二元牌序列与全部区间，并打乱询问顺序、真实编码异或输入。另有随机牌局、 $n = q = 300000$ 的全异、全同、周期序列闭式检查，以及稀疏重复大数组的短区间逐张模拟。两套驱动均在线 AC，各 41 个正式测试和 3 个额外测试全过：[动态版](#) 最慢 789 ms、452688 KB，[传统版](#) 最慢 752 ms、452492 KB。此应用仅调用区间加 1、后缀拼接、单点读取；一般有符号增量与区间求和仍由前节本地测试支持。

17.3 带修莫队：时间轴与区间状态

先离线读完操作，记录每个单点修改的下标、修改前值和修改后值。询问的时间 t 表示前 t 次修改已执行；输入中的询问本身不推进时间。同一位置的第二次修改，其旧值必须来自第一次修改后的数组，不能始终使用初始值。需要离散化时，应将全部初值与修改后值一同纳入。

调度器保持三个指针 (l, r, t) ，回调统计恰好对应当前版本的半开区间 $[l, r)$ 。前进修改 i 时写入 after，回退时写入 before。若修改点在当前区间内，必须先删除旧贡献、改值、再加入新贡献；区间外只改数组。随后移动区间端点，最后按原询问编号写答案。扩展区间先于收缩，支持空区间和互不相交区间。整个 run 结束后状态停留在排序后的最后一条询问；再次调用前须重置原数组和统计量。

左端点分块、右端点分块，再按时间排序，相邻块采用蛇形顺序。时间关键字是真实修改次数，不能把询问编号当作时间维度。令块长为 B ，询问数为 Q ，修改数为 U ，则常数时间回调下的移动量上界为

$$O\left(QB + \frac{n^2}{B} + U \min(Q, (\lfloor n/B \rfloor + 1)^2)\right).$$

另有 $O(Q \log Q)$ 排序与 $O(Q)$ 调度空间。三者同阶时取 $B \approx n^{2/3}$ ，得到 $O(n^{5/3})$ ；规模悬殊时可手动调整块长。回调需要区分左右端增删时应拆分四个方向，当前简版适合不同数、频次矩等统计。

P1903 数颜色 驱动以颜色频次数组维护不同数，真实操作顺序建立修改前后值，随后交给调度器。本地快照测试逐项检查每个时间、每个区间及修改回退状态，另以顺序 set 查询核对完整驱动。最大规模验证 133333 个元素和操作，并覆盖仅修改、仅询问与重复修改同一位置。最终简短命名版本在 [P1903 记录 297653743](#) 通过全部 13 个测试，最慢单点 546 ms、峰值内存 14.65 MB。此记录使用默认块长与不同颜色数统计；自定义块长、空区间及任意登记时间由本地测试覆盖。最快提交榜的排名及总数尚未核验。

17.4 并查集模板题的接口范围

[P3367](#) 输入点号为 1-based，驱动统一减一后调用 dsu。[记录 297668102](#) 通过全部 20 个测试，最慢单点 196 ms、峰值内存 2.27 MB。该输出验证 merge、same 及其调用的 find，不验证 size、groups；后两者已由独立标签参考和 50 万点长链测试核对。

第一部分

跨类依赖

附录 A 数论

A.1 静态模整数

模板参数必须为不超过 `INT_MAX` 的素数且 ≥ 2 。加减乘、`pow`、`inv`、除法；求逆要求非零，`pow` 指数非负。复杂度：加减乘 $O(1)$ ，幂和逆 $O(\log \text{mod})$ 。支持 `+=`、`-=`、`*=`、`/=`，返回自身引用，允许自赋值和链式运算；可作为 Fenwick 区间和的值类型。复合除法沿用非零元素、素数模数前提。

```
1 template <int mod> struct ModInt
2 {
3     int v;
4
5     ModInt(long long x = 0) : v((x % mod + mod) % mod) {}
6
7     ModInt operator+(ModInt b) const { return ModInt((long long)v + b.v); }
8
9     ModInt operator-(ModInt b) const { return ModInt((long long)v - b.v); }
10
11    ModInt operator*(ModInt b) const { return ModInt(1LL * v * b.v); }
12
13    ModInt pow(long long e) const
14    {
15        assert(e >= 0);
16        ModInt a = *this, r = 1;
17        for (; e >>= 1, a = a * a)
18            if (e & 1) r = r * a;
19        return r;
20    }
21
22    ModInt inv() const
23    {
24        assert(v);
25        return pow(mod - 2);
26    } // prime modulus
27
28    ModInt operator/(ModInt b) const { return *this * b.inv(); }
29
30    ModInt &operator+=(ModInt b) { return *this = *this + b; }
31
32    ModInt &operator-=(ModInt b) { return *this = *this - b; }
33
34    ModInt &operator*=(ModInt b) { return *this = *this * b; }
35
36    ModInt &operator/=(ModInt b) { return *this = *this / b; }
37 };
```

算法索引

AffineSegTree, [17](#)

basis_intersection, [15](#)

basis_sum_intersection, [16](#)

dsu, [4](#)

DynamicKth, [29](#)

Fenwick, [6](#)

GcdSequenceTreap, [44](#)

gp_map, [3](#)

LazySeg, [7](#)

LiChao, [19](#)

ModifiedMo, [11](#)

ModInt, [53](#)

OrderedSplay, [40](#)

OrderedTreap, [34](#)

ost, [2](#)

PersistentArray, [23](#)

PersistentDistinct, [32](#)

PersistentKth, [20](#)

PersistentRange, [26](#)

PositionBasis, [13](#)

RollbackDSU, [5](#)

rp, [1](#)

SequenceTreap, [36](#)

XorBasis, [8](#)

带修莫队, [50](#)